

Weekly 20% Learning Schedule (4×2h) — Template

This document is a downloadable, editable weekly calendar to help you plan 20% time for learning (8 hours/week) using the 4×2h strategy. Adjust times, tasks, and days to fit your work schedule. Each block includes a micro-task template and a short "Resume Note" area to minimize context switching.

Week Overview

Strategy: 4 blocks × 2 hours each (8 hours/week). Recommended: Mon, Tue, Thu, Fri.

Goal: Each block should produce one small, verifiable output (e.g., a code chunk + test, a 200-word design note, or a PR).

Weekly Calendar (editable)

Time / Day	Monday	Tuesday	Wednesday	Thursday	Friday
08:00–10:00	20% Learning Block (Deep reading / Write design note)			Copilot Experiment (spec -> code skeleton)	
10:15–12:15		Focus Block (Tangle / build / test)			
13:30–15:30					
15:45–17:45					PR / CI / Wrap-up (Write README / run CI)
Notes					

Tip: Replace the suggested blocks with times that match your calendar and move any block that collides with fixed meetings.

Micro-task Card Template (copy for each block)

Title	[Short task title]
Estimate	30min / 1h / 2h
Goal (done criteria)	e.g., unit test passes / doc 200 words / tangle OK
Preconditions	repo on dev branch, tangle script ready
Steps	1) Open doc chunk 2) Write pre/post

	3) Tangle -> build -> run test
Resume Note (end of block)	Status / Next step (2 lines)

Place one Micro-task Card inside the calendar cell or keep them in your task manager (Notion / GitHub issues).

Suggested Micro-tasks (30–60min each)

1. 1. Read a Knuth section and write a 3-sentence summary
2. 2. Write pre/postcondition for one function (200 words)
3. 3. Break an existing .c function into a named chunk and tangle
4. 4. Add a minimal usage example to the literate doc
5. 5. Use Copilot to generate a function skeleton inside the doc
6. 6. Write a unit test (host) and run in CI
7. 7. Draft a design-decision note (trade-offs, 3 lines)
8. 8. Add a hardware register map to appendix
9. 9. Create or test Makefile tangle step
10. 10. Refactor complex code into smaller chunks
11. 11. Add an FAQ entry for a common misuse
12. 12. Draw an ASCII statechart and include it in the doc
13. 13. Write a Resume Note and attach to the issue
14. 14. Make CI run tangle as a pre-build step
15. 15. Write a 500-word short tutorial for a pattern
16. 16. Do a 30-min peer review and collect feedback

Weekly Review (30 minutes)

Use this template at the end of the week to track progress and plan next week:

What I learned
What I built (outputs)
Blocked by / Issues
Next week focus

KPI suggestions: at least 1 verifiable output per week (code chunk + test or a 200-word technical note). Every 6 weeks, prepare a shareable artifact (blog, repo, internal talk).